

CS-600: Software Design and Development

3-2 Activity: Integrating Security Frameworks in Software Testing

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

May 4, 2026

This activity strengthened the rare-books catalog application by adding security controls that were missing from the original implementation and test plan. The original test plan focused mainly on functional behavior, such as confirming that endpoints were accessible, that ID values were unique, that fields were not null, and that duplicate records were not present. Those checks are still useful, but they are not enough for an application that can expose personally identifiable information. Because the audit found missing encryption, authentication, and weak data handling, I revised the test plan, so it now checks both normal application and security behavior.

The revised test plan addresses vulnerabilities by adding tests for encrypted communication, authentication, authorization, password handling, endpoint access, and secure data access. Testing HTTPS on port 8443 confirms that information is encrypted while it travels between the client and the application. This matters because data in flight can be intercepted if it is transmitted through plain HTTP. The test plan also checks that the public version endpoint stays available for basic version verification, while protected endpoints such as `/api/v1/names` require valid login credentials before database records are returned.

The authentication tests strengthen reliability by confirming that unauthenticated users cannot access protected data and that valid users can access the application only with correct credentials. These checks help catch configuration mistakes early, such as accidentally leaving a data endpoint public or allowing invalid credentials to succeed. The revised test plan also includes vulnerability-focused tests for null values, duplicate records, and expected database results. These tests are important because security improvements should not break existing functionality. The application must remain reliable while also becoming safer.

Spring Security is proper for this application because it integrates directly with Spring Boot and offers a standard way to manage authentication, authorization, password encoding, and security filters. Instead of creating custom login and access-control logic from scratch, the application can use a proven framework that is widely used in Spring applications. In this assignment, Spring Security uses HTTP Basic authentication with an in-memory demonstration user. The password is encoded with BCrypt, which is safer than storing a plain-text password. In a production environment, this user should be replaced with accounts stored in a database or identity provider, and secrets should be moved into environment variables or a secure secret manager.

The updated configuration also supports encryption of data in flight through HTTPS using the provided PKCS12 keystore. This is important for an internal catalog application because internal systems can still expose sensitive information if traffic is not protected. Requiring HTTPS and authentication together creates a stronger security baseline. Overall, Spring Security is a good fit because it allows meaningful protection to be added without changing the public API structure. The endpoints still work as expected, but access to sensitive information is now controlled and transmitted through an encrypted channel.

